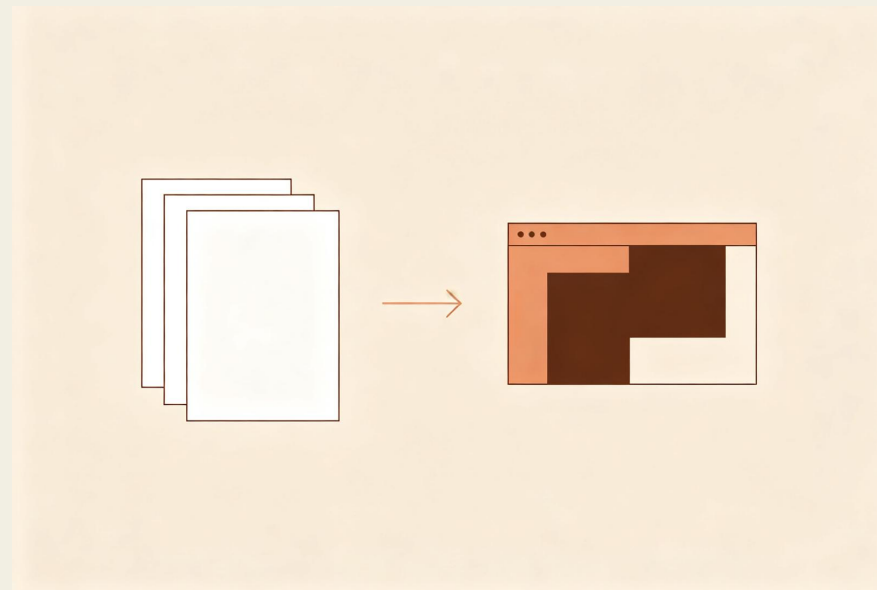


OpenSpec

SPEC-DRIVEN DEVELOPMENT

规格驱动开发 (SDD) 与 OpenSpec

先定义 WHAT，再让 AI 完成 HOW
AI 编程时代的软件工程新范式



本讲内容

一条主线：让 AI 从「猜」到「照单执行」

01



为什么需要 SDD

从 Vibe Coding 的失控说起：AI 时代为什么需要「契约」

02



什么是 SDD

规范成为唯一事实来源：先 WHAT，再 HOW

03



哪些工具可以实现 SDD

工具全景与选型：OpenSpec / Spec-Kit / Kiro / TESSL

04



为什么推荐 OpenSpec

轻量、开源、Brownfield 优先，30+ 工具原生集成

05



怎么用 OpenSpec 实现 SDD

从安装初始化到 propose → apply → archive 的完整实战

01

为什么需要 SDD

当 AI 开始大规模生成代码，需求对齐成了最大的瓶颈——这正是 SDD 要解决的问题。

Vibe Coding 的问题：需求只活在聊天记录里



需求易失

需求只存在于聊天上下文里，会话一关就丢，无法复用、评审与追溯。



AI 自由发挥

缺少明确契约，AI 只能靠猜：漏掉关键功能、添加冗余逻辑、破坏既有架构。



返工代价高

方向错了要等代码跑起来才发现，返工成本随规模成倍放大。

失控回路：每一轮都要为「猜错」付全款



根源：从「人脑补齐」到「AI 盲猜」

维度	传统开发（人写代码）	AI 编程
需求载体	会议纪要、口头沟通、人脑记忆	聊天记录，易失且不可评审
上下文补齐	靠程序员经验与长期记忆	AI 缺少项目上下文，只能猜
评审对象	代码 + 设计文档	生成的代码，难以逐行审查
错误暴露	开发与测试阶段发现	常在集成或交付后才暴露

结论：口头需求是「无法评审的输入」；我们需要一个中间层——把需求变成人机都读得懂、可评审、可测试的规范。

02

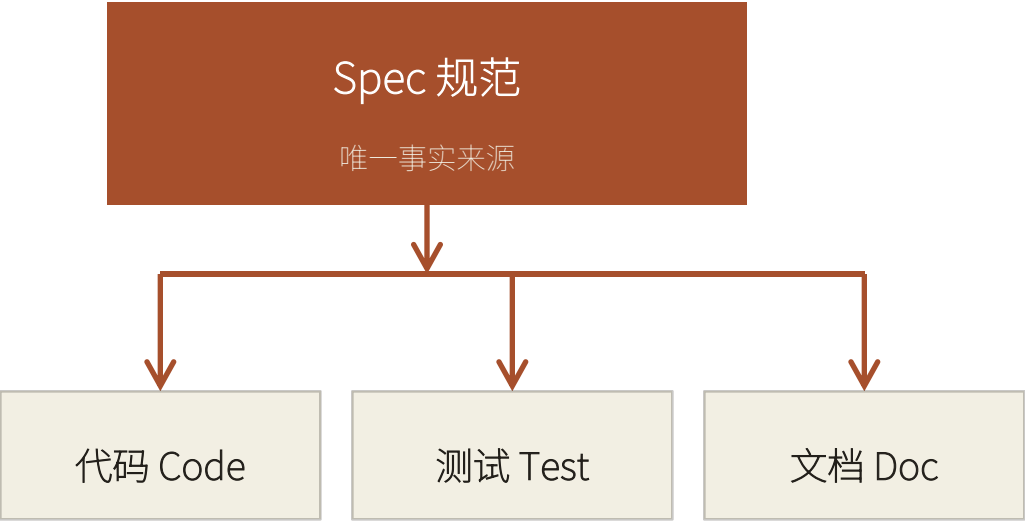
什么是 SDD

把规范作为唯一事实来源，让代码成为它的派生产物——
先定义 WHAT，再让 AI 完成 HOW。

SDD：规范是唯一事实来源，代码是派生产物

核心定义

先把「需求、约束、设计意图、验收标准」结构化为**规范 (Spec)**，再以规范为**唯一事实来源** (Single Source of Truth) 开展计划、实现、测试与评审。
代码，只是规范的**派生产物**。



WHAT 与 HOW 分离

先定义「要什么」，再让 AI 或工程师去「怎么做」。

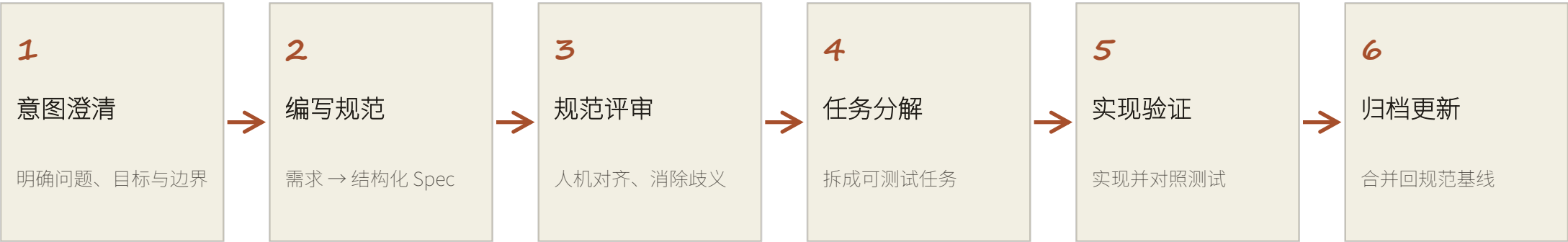


规范可执行、可评审

不再是「写完没人看的文档」，而是可测试、可追溯的契约。

改规范 → 重新实现与验证；改代码 → 需要同步回规范。

SDD 的标准 workflow



贯穿全程：规范是唯一的「真相源」。写代码前先对齐，写代码后向规范同步——规范与代码同仓库、同版本、同评审。



人机协作

AI 起草规范，人来评审把关。



任务可测试

每个任务对应可验收的交付单元。



变更可追溯

归档即更新「活规范」。

一份可执行的规范：Requirement + Scenario

Requirement: 用户认证
系统 **SHALL** 在登录成功后签发 JWT。

Scenario: 凭证有效
- **WHEN** 用户提交有效凭证
- **THEN** 系统返回 JWT

规范的写法规则



需求 = 确定性断言

用 SHALL / MUST 描述系统必须满足的行为，不留模糊空间。



场景 = 验收标准

每个需求至少一个 Scenario：WHEN 触发条件，THEN 期望结果。



规范可被机器校验

用 validate 检查格式与结构 → 可执行、可测试、可追溯。

SDD vs 传统流程：从「写完没人看」到「可执行契约」

维度	传统开发	SDD
规范地位	文档，写完容易过时	唯一事实来源，随变更持续更新
评审时机	评审代码本身	先评审规范，再进入实现
变更代价	改需求 → 人工改代码	改规范 → 重新实现与验证
人机协作	需求靠人来转译	规范同时被人与 AI 读取
测试依据	靠经验补用例	场景（Scenario）即验收用例

SDD 让「需求 → 代码」从「模糊接力」变成「契约驱动」

03

哪些工具可以实现 SDD

把 SDD 落地成工作流的工具已经形成生态——
从开源框架到商业平台，各有擅长。

SDD 工具生态全景

OpenSpec 开源 · MIT 轻量规范框架，AI 编码助手 SDD，Brownfield 优先。	Spec-Kit 开源 七步瀑布式工作流，全新项目 0→1 场景强。	Kiro.dev 商业 会话式 AI 编程与规范管理，按 feature 组织。
Tessl 商业 「规范即源码」，代码由规范直接生成。	SpecStory 免费工具 把 Cursor 开发会话自动沉淀为规范文档。	AGENTS.md 轻量约定 仓库内指令文件约束 AI 行为，零依赖起步。

归纳：两类思路 —— 规范锚定（spec-anchored）vs 规范即源码（spec-as-source）

工具选型对比

对比项	OpenSpec	Spec-Kit	Kiro.dev	Tessl
规范定位	规范锚定	规范锚定	会话式管理	规范即源码
最佳场景	已有项目 1→n	全新项目 0→1	团队协作	规范驱动生成
变更追踪	现状/提案分离	单一流程	分散多目录	规范为源码
工具集成	30+ 工具	有限	平台绑定	平台绑定
开源情况	开源 · MIT	开源	商业	商业

选型建议： 全新项目可看 Spec-Kit；要落地真实项目、尤其是改造既有系统，OpenSpec 更合适。

04

为什么推荐 OpenSpec

轻量、开源、可落地——

OpenSpec 用最低的成本把 SDD 变成团队的日常习惯。

OpenSpec: 轻量、开源的 SDD 框架

它是什么

OpenSpec 是轻量、可配置、开源的规格驱动开发框架：用规范描述要构建的内容，让团队和编码 Agent 在写代码前**达成一致**，并在演进中持续对齐。

66.7k

GitHub Stars

265k+

月活跃开发者

MIT

开源协议

0

需要的 API Key

数据来源：openspec.dev (2026-09 访问)

核心特点



轻量

纯 Markdown + 一个 CLI，无需服务端。



Brownfield 优先

现状 specs/ 与提案 changes/ 分离，改造既有系统也不怕。



30+ 工具兼容

Claude Code、Cursor、Codex、Copilot 等原生集成。



可审计可归档

每次变更独立文件夹，proposal/tasks/delta 同处存放。

OpenSpec 与其他方案：差异一目了然

04 · 为什么推荐 OpenSpec

对比项	OpenSpec	Spec-Kit	无规范（纯聊天）
规范定位	规范锚定，真相源	规范锚定	无
适合场景	新功能 + 改造既有	全新项目 0→1	原型探索
变更追踪	specs/ + changes/ 分离	单一流程文件	聊天记录
人机对齐	写码前对齐，可评审	写码前对齐	边聊边猜
成本门槛	开源 MIT · 无需 Key	开源	零成本但不可控

结论：在「真实项目落地」这条线上 OpenSpec 综合最优——既能 0→1，也能从容地 1→n。

05

怎么用 OpenSpec 实现 SDD

三步走：安装初始化 → 跑通 propose / apply / archive 工作流 → 把「先对齐、再实现」沉淀成日常习惯。

第一步：安装与初始化

```
~/project $  
  
$ npm install -g @fission-ai/openspec@latest  
$ openspec --version  
  
$ cd my-project  
$ openspec init
```

终端负责安装与初始化；真正的 slash 命令在 AI 聊天框里执行。

初始化做了什么



选择你的 AI 工具

Claude Code / Cursor / Codex / Copilot...



自动配置命令

生成 slash 命令配置，并写入 AGENTS.md 交给 Agent。



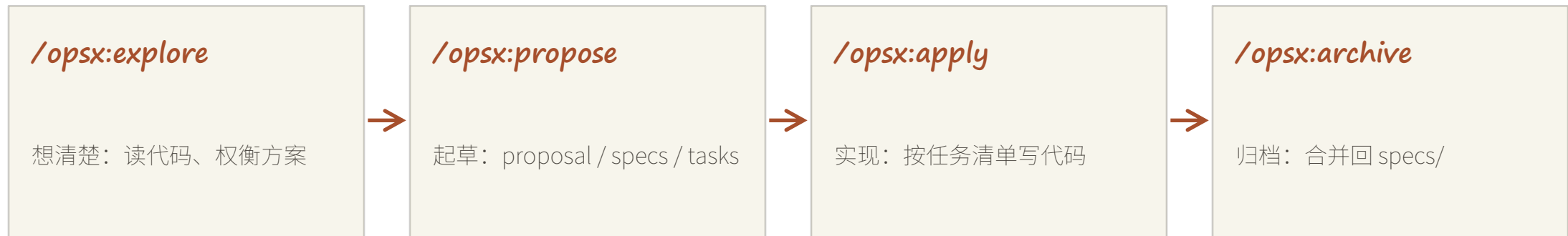
创建 openspec/ 目录

项目上下文、规范、变更各就各位。

```
openspec/  
├─ project.md 项目上下文  
├─ specs/ 当前规范（真相）  
└─ changes/ 变更提案
```

第二步：用 slash 命令跑通完整 workflow

05 · 怎么用 OpenSpec 实现 SDD



终端里常用的 openspec 命令

```
$ openspec list  
$ openspec show <change>  
$ openspec validate <change>  
$ openspec archive <change> --yes
```



终端 · openspec 命令

list / show / validate / archive



聊天框 · */opsx:* 命令

explore / propose / apply / archive

一个完整例子：为系统添加双因素认证（2FA）

AI 自动生成的文件

```
openspec/changes/add-2fa/  
├─ proposal.md 为什么与做什么  
├─ tasks.md 实现任务清单  
├─ design.md 技术决策（可选）  
├─ specs/auth/  
└─ spec.md delta 变更
```

你不手写这些文件——对 AI 说一句话，它就按规范结构生成；你负责评审。

spec delta：新增需求 + 验收场景

```
### Requirement: 双因素认证  
系统 MUST 在登录时要求第二重验证。  
  
#### Scenario: OTP 校验  
- WHEN 用户提交有效凭证  
- THEN 触发 OTP 挑战
```

tasks.md：可执行任务清单

```
## 1. 数据库  
- [ ] 1.1 用户表增加 OTP 密钥列  
## 2. 后端  
- [ ] 2.1 增加 OTP 生成接口  
- [ ] 2.2 登录流程要求 OTP
```

在团队里落地 OpenSpec

最佳实践

 每个变更 = 一个分支 + 一个 PR

先评审规范，再写代码。

 规范与代码同仓库、同版本、同评审

变更完成后及时归档，回到一致状态。

 把 validate 接入 CI

规范结构不合格直接拦截合并。

 从新功能试点，逐步沉淀「活规范」

不追求一步到位，先跑通一个小功能。

常用命令速查

```
$ openspec list
$ openspec show <change>
$ openspec validate <change>
$ openspec archive <change> --yes
$ openspec update
```

更换 AI 工具后运行 openspec update，刷新 slash 命令。

小结：让 AI 从「猜」到「照单执行」



规范 = AI 时代的人机契约

需求不再是易失的聊天记录，而是可评审、可测试、可追溯的规范。



OpenSpec 是最低成本的落地路径

开源、轻量、无需 API Key，30+ 工具原生兼容。



从一个小功能开始

先对齐、再实现，把 SDD 变成你的日常习惯。

课后动手 · 资源

openspec.dev

github.com/Fission-AI/OpenSpec

openspec.dev/docs

Build the right thing, and build it right.